FlexQL Design Document

1 Repository

https://github.com/Somak-2001/FlexQL.git

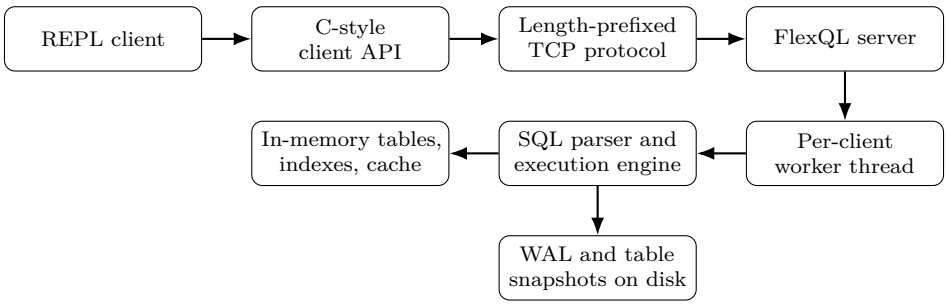
2 Overview

FlexQL is a lightweight SQL-like client/server database driver implemented fully in C++. The system is split into:

- A TCP database server that parses and executes SQL-like commands
- A C-style client API layer that communicates with the server
- An interactive REPL built on top of the client API

The implementation supports `CREATE TABLE`, `INSERT`, `SELECT`, single-condition `WHERE`, and `INNER JOIN`.

3 System Architecture



4 Storage Design

4.1 Row-Major Storage

The database uses a row-major in-memory layout:

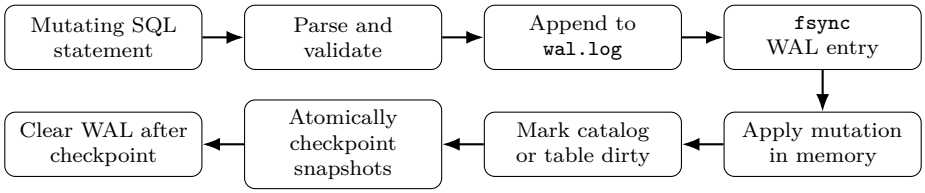
- Each table stores a vector of rows
- Each row stores a vector of string values aligned with the schema
- This makes inserts simple and keeps row reconstruction cheap during `SELECT *`

4.2 Durable Storage

FlexQL persists data in a small WAL-plus-snapshot layout under `flexql_data/`.

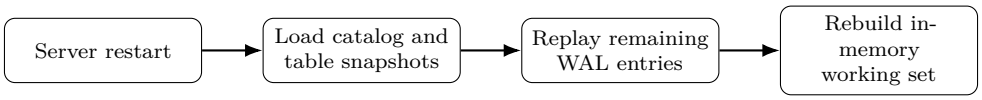
- `catalog.txt` stores table schemas
- `tables/<TABLE>.rows` stores checkpointed live rows
- `wal.log` stores mutating SQL statements that have been synced but not yet checkpointed

The write path is:



1. Parse and validate the statement
2. Append the normalized mutating SQL statement to the WAL and `fsync` it
3. Apply the mutation in memory
4. Mark the affected table or catalog as dirty
5. Rewrite dirty catalog/table snapshots atomically at checkpoint time
6. Clear the WAL only after checkpoint completion

On restart, the engine loads the catalog and table snapshots first, then replays any remaining WAL entries. This means a crash after the WAL append but before checkpointing is recoverable: the snapshot may be older, but the synced WAL entry is replayed to reconstruct the committed state.



Disk is the durable source of truth. RAM is used only as the active working set: table rows, primary-key hash indexes, and cached `SELECT` results are loaded and maintained in memory so query execution avoids disk reads on the hot path. This improves assignment-scale latency and keeps the implementation simple, but it means very large tables require enough RAM for their active rows and indexes. The snapshots and WAL make the in-memory state reconstructible after restart, while the LRU query cache remains an optimization only and can be discarded at any time.

The main persistence trade-off is checkpoint frequency. Checkpointing after every row would minimize WAL replay work but would add heavy disk I/O. FlexQL instead syncs each mutating statement to the WAL and checkpoints dirty snapshots in batches, which reduces write amplification while preserving recoverability.

4.3 Schema Representation

Each table keeps:

- A list of columns with name, type, and primary-key metadata
- A hash map from column name to column index
- A row container

Supported types:

- `DECIMAL`
- `VARCHAR`

Type checks are performed during insertion. Expiration timestamps are stored separately as absolute epoch seconds so TTL handling remains independent of the table's supported column types.

5 Indexing Method

Primary indexing is implemented as a hash map:

- Key: primary-key value
- Value: row offset inside the table storage

This design makes exact-match lookups on the indexed column efficient and keeps insert overhead low.

Current assumption:

- At most one primary-key column per table

6 Query Execution

6.1 CREATE TABLE

- Parses schema metadata
- Verifies duplicate column names are not present
- Registers the table in the in-memory catalog

6.2 INSERT

- Validates every row in the statement against schema width and type rules
- Enforces unique primary-key values against both existing rows and rows inside the same batch
- Supports multi-row `INSERT INTO ... VALUES (...), (...), ...`
- Stores an optional absolute expiration timestamp for each inserted row

6.3 SELECT

- Supports `SELECT *` and projected columns
- Supports a single `WHERE` predicate
- Supports `INNER JOIN ... ON ...`

6.4 WHERE Restriction

- Only one condition is supported
- No `AND` or `OR`

7 Expiration Handling

Each row stores an optional expiration timestamp in epoch seconds.

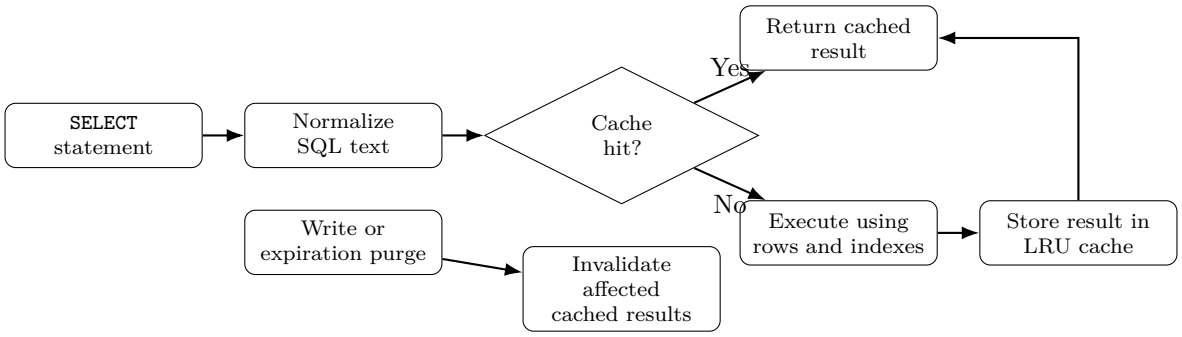
Supported insertion forms:

- `TTL <seconds>`
- `EXPIRES AT 'YYYY-MM-DD HH:MM:SS'`

Expired rows are removed lazily during future access to the table. This avoids doing background cleanup work while still preserving correctness.

8 Caching Strategy

FlexQL uses an LRU result cache for `SELECT` queries.



- Cache key: normalized SQL text
- Cache value: complete query result including columns and rows
- Capacity: 128 query results

The cache is invalidated whenever:

- A table is created
- A row is inserted
- Expired rows are purged

This keeps cached results correct after writes.

9 Multithreading Design

The server follows a thread-per-client model:

- The listening thread accepts incoming TCP connections
- Each client connection is handled by a worker thread
- Shared database state is protected with `std::shared_mutex`
- The query cache has its own mutex

This allows multiple clients to issue queries concurrently while preventing race conditions on shared structures.

10 Networking Design

Client and server communicate through a simple framed protocol:

- Every message is prefixed with a 32-bit payload length
- The client sends one SQL statement per request
- The server replies with result headers, row frames, and a terminal `END` frame

This avoids ambiguity from newline-based parsing and makes binary-safe message transfer straightforward.

11 Performance Notes

The implementation is designed to scale reasonably for assignment workloads through:

- Row-major storage for cheap appends
- Hash-based primary indexing
- LRU caching for repeated reads
- Batch insert parsing to reduce client/server round trips
- WAL-first persistence with batched checkpoints to reduce disk write amplification
- Lazy expiration cleanup

The included benchmark utility can be used to measure:

- Insert throughput
- Point-select latency

Local performance results measured on April 7, 2026:

- TCP unit test: `./benchmark_bin -unit-test` passed 21/21 against `./server` on `127.0.0.1:9000`
- Benchmark wrapper, 1M rows: `./benchmark 1000000` inserted 1,000,000 rows in 0.083905 sec, or 11,918,240 rows/sec; budget check passed against the 2.0 sec target

- Benchmark wrapper, 10M rows: `./benchmark 10000000` inserted 10,000,000 rows in 0.836931 sec, or 11,948,416 rows/sec; budget check passed against the 20.0 sec target
- Read-heavy check outside the insert benchmark: 100 repeated `SELECT*FROMCACHE_BENCH_20260407WHEREID=42;` requests over the TCP REPL completed in 8.15 sec end-to-end, about 81.5 ms/request including client, protocol, and table-printing overhead
- Concurrent-client check outside the insert benchmark: 4 REPL clients issuing 25 cached point selects each completed 100 total selects in 1.98 sec, with no client error output

12 Compilation Instructions

Normal local development binaries are built with `make`. This produces `./flexql_server`, `./flexql_repl`, `./flexql_benchmark`, and `./smoke_test`.

```
make
```

The professor benchmark wrapper workflow uses `sh compile.sh` instead. That script builds the benchmark-compatible binaries and wrapper commands named `./server` and `./benchmark`.

```
sh compile.sh
```

13 Execution Instructions

Professor benchmark wrapper workflow, Terminal 1:

```
sh compile.sh
./server
```

Professor benchmark wrapper workflow, Terminal 2:

```
./benchmark --unit-test
./benchmark 1000
```

Manual REPL workflow, Terminal 1:

```
make flexql_server flexql_repl
./flexql_server
```

Manual REPL workflow, Terminal 2:

```
./flexql_repl 127.0.0.1 9000
```

Exit the REPL with:

```
.exit
```

14 Verification

Local verification completed for:

- Build success with `make`
- SQL parser and engine smoke test with batch `INSERT`, `SELECT`, `WHERE`, `TTL`, and `INNER JOIN`
- Restart persistence test using a fresh engine instance after on-disk checkpointing
- TCP client/server round-trip validation with `./benchmark_bin -unit-test`: 21/21 tests passed over `127.0.0.1:9000`